

Универзитет „Св. Кирил и Методиј“ во Скопје
**ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ И
КОМПЈУТЕРСКО ИНЖЕНЕРСТВО**

Дипломска работа

Изработка на робот управуван преку систем за виртуелна реалност

Ментор:

проф. д-р Андреа Кулаков

Изработил:

Јован Николов 192067

Скопје, Јуни 2026

Contents

Апстракт	3
1. Вовед	4
1.1 Мотивација и цел	4
1.2 Структура на трудот	4
2. Технологии и алатки	5
2.1 Unity 6 и Universal Render Pipeline	5
2.2 SteamVR	5
2.3 Raspberry Pi 4B	5
2.4 PCA9685 PWM контролер	5
2.5 UDP мрежна комуникација	6
2.6 RTSP видео проток	6
2.7 Tinkercad и 3Д печатење	6
3. Хардверска архитектура	7
3.1 Физички опис на роботот	7
3.2 Серво мотори и степени на слобода	8
3.3 Мапирање на PCA9685 канали	8
3.4 PWM конверзија на агол	10
3.5 3Д печатење на делови	11
4. Системска архитектура	12
4.1 Преглед на системот	12
4.2 Тек на податоци	12
4.3 Unity сцена и компоненти	13
5. VR клиент, Unity имплементација	14
5.1 Следење на VR контролери (ControllerTracker)	14
5.1.1 Позиционирање на рамото	14
5.2 Инверзна кинематика за раце	14
5.2.1 Помошни функции	15
5.2.2 Флексија/екстензија на рамо (Pitch) – Канал 0/6	15
5.2.3 Абдукција/адукција на рамо (Roll) – Канал 1/7	15
5.2.4 Медијална/латерална ротација на рамо (Yaw) – Канал 2/8	15
5.2.5 Свиткување на лакт – Канал 3/9	16
5.2.6 Ротација на зглоб – Канал 4/10	16

5.2.7	Зафат на дланка – Канал 5/11.....	17
5.2.8	Калибрациско поместување.....	18
5.3	Следење на главата	18
5.4	Управување со комуникацијата.....	19
5.4.1	Безбедносен механизам.....	19
5.5	Дисплеј за статус	19
5.6	Видео проток.....	20
6.	Серверска имплементација (Raspberry Pi)	21
6.1	UDP сервер	21
6.2	Главна јамка за обработка на податоците	21
7.	Мрежна комуникација.....	23
7.1	UDP протокол и формат на пораки	23
7.2	Временска анализа и задоцнување	23
7.3	RTSP видео проток	23
8.	Тестирање и резултати	25
8.1	Методологија на тестирање	25
8.2	Резултати	26
8.3	Предизвици и ограничувања.....	26
9.	Заклучок	27
9.1	Постигнати резултати	27
9.2	Насоки за понатамошна работа	27
	Литература	29
	Додаток А: Преземени 3Д модели	29
	Додаток Б: Хардверски компоненти	30
	Додаток В: Изворен код и демонстрација	30

Апстракт

Оваа дипломска работа го опишува развојот на систем за далечинско управување со човеколик робот преку технологијата за виртуелна реалност (ВР). Системот овозможува интуитивно пренесување на движењата на рацете и главата на корисникот врз физичкиот робот во реално време, обезбедувајќи природен телепресентен начин на управување без потреба од специјализирана обука.

Клиентскиот дел е изработен во Unity 6 Engine со SteamVR интеграција. ВР слушалките и контролерите ги следат движењата на корисникот, кои преку алгоритми за инверзна кинематика (ИК) се конвертираат во 14 серво вредности, по 6 за секоја рака (рамо Pitch/Roll/Yaw, лакт, зглоб и дланка) и 2 за главата (вртење по вертикална и хоризонтална оска). Овие вредности се испраќаат со фреквенција од 20 Hz преку UDP протокол до Raspberry Pi компјутерот.

Серверскиот дел е Python скрипта на Raspberry Pi 4B која го прима UDP пакетот, ги парсира 14-те агли и ги претвара во PWM сигнали преку PCA9685 PWM контролер поврзан преку I2C протокол, управувајќи со серво моторите на роботот. За повратна информација, видео сигналот од камерата на роботот се пренесува преку RTSP протокол до ВР опкружувањето, каде што се прикажува на екранот на слушалките за виртуелна реалност. Во видеото на екранот е интегриран инфо приказ за тековната состојба на системот.

Системот е потполно функционален и успешно ги исполнува поставените цели: следење на движењата во реално време со задоцнување од 160 до 280 ms во оптимални услови, заштита на хардверот преку механизам за мазно движење со интерполација од 120° во секунда и визуелна повратна информација преку видео проток со резолуција 640x480 пиксели и 30 слики во секунда.

Клучни зборови: виртуелна реалност, инверзна кинематика, роботика, SteamVR, Unity, Raspberry Pi, PCA9685, UDP, RTSP

1. Вовед

1.1 Мотивација и цел

Управување со роботи на далечина (телеоперација) претставува активна истражувачка и инженерска област со широка примена: хируршки роботи, инспекција на опасни средини, истражување на вселената, образование, рехабилитација и забава. Традиционалните пристапи за телеоперација се базираат на дводимензионален монитор и контролер со копчиња, при што операторот губи значаен дел од просторниот контекст, длабочинскиот увид и интуитивноста на управувањето.

Виртуелната реалност (ВР) нуди суштинско решение за овој проблем преку имерзивна тридемензионална средина и следење на природните движења на телото. Наместо да научи нов контролен интерфејс, операторот едноставно ги врши движењата кои сака да ги изврши роботот – рацете и главата се следат директно и се пренесуваат врз човеколиката структура.

Основна цел на оваа дипломска работа е да го истражи и имплементира системот кој:

- Ги следи природните движења на корисникот во ВР просторот (позиција и ориентација на двете раце и главата).
- Ги пресметува потребните серво агли преку алгоритми за инверзна кинематика.
- Ги пренесува командите во реално време преку мрежа до физичкиот робот.
- Обезбедува видео како повратна информација за ориентација на операторот.
- Го заштитува хардверот преку безбедносни механизми.

1.2 Структура на трудот

Трудот е организиран по редоследот:

- Поглавје 2 ги претставува технологиите и алатките употребени во системот.
- Поглавје 3 ја опишува хардверската архитектура на роботот како серво моторите, PCA9685 контролерот, Raspberry Pi и производството на структурните делови преку 3Д печатење.
- Поглавје 4 ја претставува целокупната системска архитектура и текот на податоци.
- Поглавје 5 ги детализира алгоритмите и имплементацијата на Unity VR клиентот, вклучувајќи ја инверзната кинематика.
- Поглавје 6 ја опишува серверската имплементација на Raspberry Pi.
- Поглавје 7 го опишува мрежниот протокол, комуникацискиот слој и временската анализа.
- Поглавје 8 ги претставува резултатите и предизвиците.
- Поглавје 9 содржи заклучни согледувања и насоки за понатамошна работа.

2. Технологии и алатки

2.1 Unity 6 и Universal Render Pipeline

Unity 6 (верзија 6000.0.38f1) [2] е моторот за игри избран за клиентскиот дел на системот. Unity нуди богат екосистем за ВР развој, вграден мотор за физика, поддршка за C# скриптирање и вградени команди за SteamVR. Universal Render Pipeline (URP) [2] обезбедува рендерирање прилагодено за ВР уреди, со оптимизирана употреба на GPU ресурсите. Изборот на Unity 6 е мотивиран со имплементираната поддршка за SteamVR, документацијата за ВР развој и брзото C# прототипирање.

2.2 SteamVR

SteamVR претставува слој на апстракција на ВР хардвер развиен од Valve Corporation. Тој овозможува единствено API за следење на контролери, слушалки и базни станици без оглед на производителот. SteamVR Unity Plugin ги обезбедува компонентите `SteamVR_Behaviour_Pose` за следење на позата (позиција и ротација), `SteamVR_Action_Single` за аналогни влезови (искористено за сензор за стисок на дланката) и `SteamVR_Action_Boolean` за дигитални копчиња. [1][H3]

2.3 Raspberry Pi 4B

Raspberry Pi 4 Model B [H1] е одбран за серверски дел поради поддршката за I2C интерфејс (потребен за PCA9685), мрежната поддршка (Ethernet, WiFi), Linux OS (Raspberry Pi OS) со поддршки за Python3, V4L2 видео API [10], ниската цена и широката достапност. [3] Raspberry Pi 4 ги извршува Python скриптата за примање на податоците преку UDP и препраќање на PCA9685, и RTSP видео серверот паралелно, управувајќи со сите 14 серво канали во реално време и за праќање на видео проток преку мрежата.

2.4 PCA9685 PWM контролер

PCA9685 [H2] е 16-канален I2C PWM/Servo контролер развиен од NXP. Тој комуницира со Raspberry Pi преку I2C протоколот (SCL/SDA линии) на стандардна адреса 0x40. Контролерот поседува 16 независни PWM канали (14 искористени), 12-битна PWM резолуција (0-4096), фреквенција на работа од 50 Hz и опсег на пулсот 500-2500 μ s. Употребата на посебен PWM контролер го ослободува Raspberry Pi од генерирање на прецизни временски сигнали, критична задача за истовремено управување на голем број серво мотори. [4]

2.5 UDP мрежна комуникација

User Datagram Protocol (UDP) е избран наспроти TCP поради следните карактеристики: нема overhead за воспоставување врска, пакетите не чекаат потврда (ACK), ниско и предвидливо задоцнување и едноставна имплементација преку UdpClient во C# и socket во Python. [11] Загубата на пакетите е прифатлива, при 20 Hz испраќање, загубениот пакет се надоместува за само 50 ms. Недостатокот на UDP каде што немаме гаранција за испорака на податоците е толериран поради краткиот период меѓу пакетите.

2.6 RTSP видео проток

Real Time Streaming Protocol (RTSP) [9] обезбедува проток на видео сигналот од камерата на роботот до Unity опкружувањето. На страната на Raspberry Pi, v4l2rtspserver го чита видео уредот и го пренесува на порта 8554. На Unity страната, сигналот прво се прима преку OBS Studio, кое го декодира RTSP протокот и го изложува како виртуелна веб камера на оперативниот систем. Unity потоа ја чита виртуелната веб камера преку стандардниот WebCamTexture API и ги прикажува рамките директно на Texture2D нанесена на виртуелен екран во VR просторот.

2.7 Tinkercad и 3Д печатење

Tinkercad е бесплатна CAD алатка основана на веб-прелистувач развиена од Autodesk, избрана за дизајнирање на структурните делови на роботот. Алатката има интуитивен интерфејс основан на конструктивна геометрија и овозможува брзо прототипирање на механички делови без потреба од напредно CAD знаење. Tinkercad директно извезува STL датотеки, кои понатаму се испраќаат на 3Д печатачот.

3Д печатењето беше одбрано за производство на физичките делови на роботот поради неколку клучни предности: можноста за брза итерација на дизајнот, ниската цена на материјалите во споредба со CNC обработка или комерцијални компоненти и можноста за производство на прецизно прилагодени носачи и конектори на серво моторите.

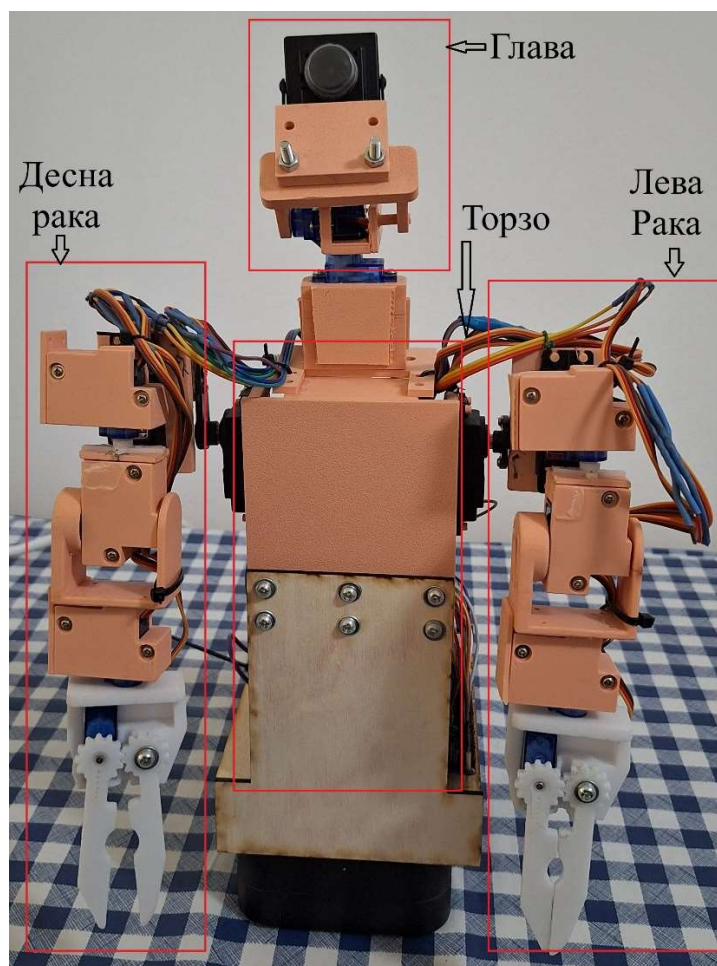
Основата на роботот е изработена од дрво, ласерски пресечено спрема 2Д дизајни изработени во Tinkercad.

3. Хардверска архитектура

3.1 Физички опис на роботот

Роботот е човеколик стационарен/торзо тип конфигурација составен од три главни дела: торзо (рамка носач), две раце и глава. Торзото ги носи двете раце и главата и е фиксирано – нема мобилна база и нозе. Дизајнот е насочен кон задачи на манипулација со рацете и гледање во просторот преку движење на главата, имитирајќи ја горната половина на човечкото тело.

Двете раце се симетрично поставени на торзото, секоја со 5 степени на слобода плус зафатот на дланката (вкупно 6 акциски степени на слобода по рака). Главата е монтирана на врвот на торзото со две оски на движење. Роботот е опремен со камера на главата за видео проток назад кон операторот.



Слика 1. Физички изглед на роботот

3.2 Серво мотори и степени на слобода

Секоја рака ги поседува следните степени на слобода, кои заедно овозможуваат тридемензионално позиционирање на зглобот и ориентација на раката:

Степен на слобода	Зглоб	Движење
1	Рамо наклон напред-назад (Pitch)	Флексија / екстензија
2	Рамо валање / странично накосување (Roll)	Абдукција / адукција
3	Рамо свртување лево-десно (Yaw)	Медијална / латерална ротација
4	Лакт	Свиткување на лакотот
5	Зглоб	Ротација на зглобот
6	Дланка	Зафат / пуштање

Главата поседува два степени на слобода:

Степен на слобода	Движење	Опис
1	Панорама (Pan)	Ротација лево-десно
2	Навалување (Tilt)	Навалување горе-долу

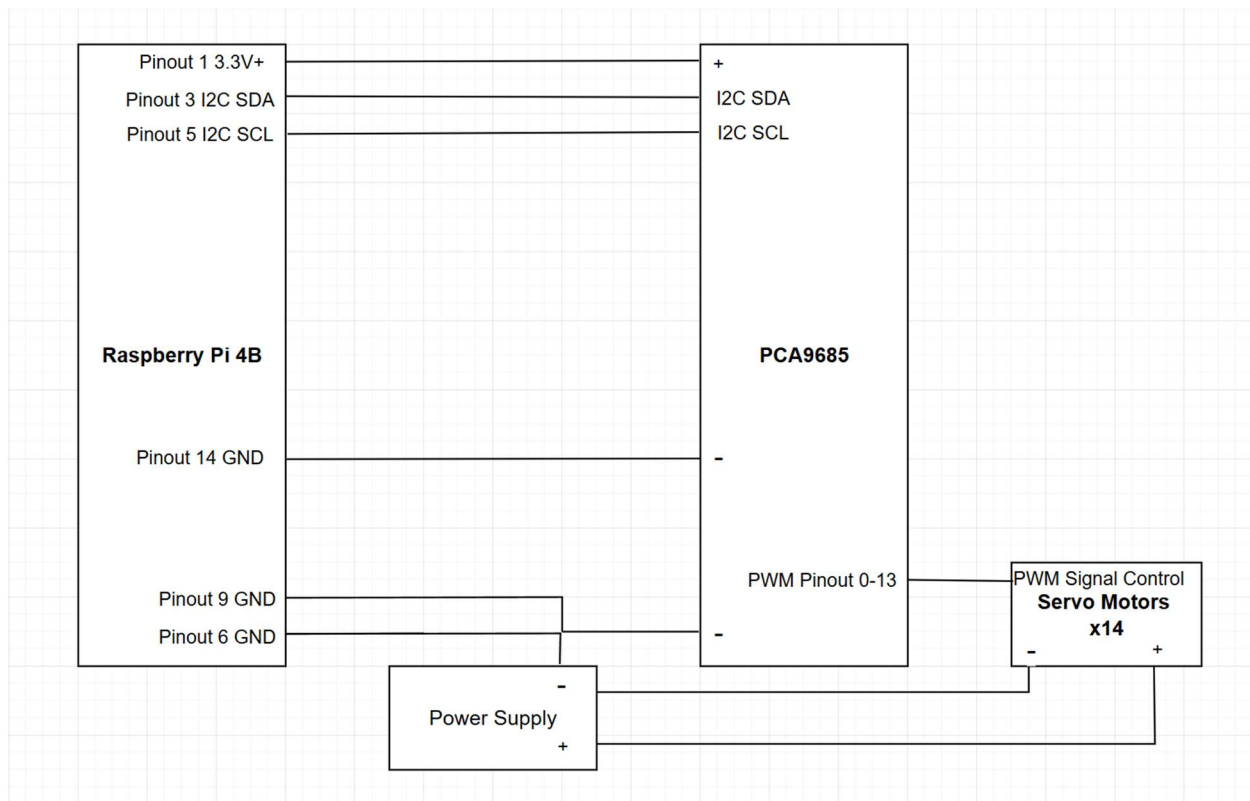
Вкупен број на серво канали: $6 \times 2 \text{ раце} + 2 \text{ (глава)} = 14 \text{ канали}$

3.3 Мапирање на PCA9685 канали

Каналите на PCA9685 контролерот се доделени на следниов начин, заедно со безбедните позиции кон кои роботот се враќа при активирање на безбедносниот механизам:

Канал	Зглоб	Безбедна позиција	Тип на серво мотор
0	Десно Рамо наклон напред – назад (Pitch)	90°	MG 996R
1	Десно Рамо валање / странично накосување (Roll)	160°	MG 996R

2	Десно Рамо свртување лево- десно (Yaw)	90°	SG 90
3	Десен Лакт	60°	SG 90
4	Десен Зглоб	130°	SG 90
5	Десна Дланка	170°	SG 90
6	Лево Рамо наклон напред – назад (Pitch)	90°	MG 996R
7	Лево Рамо валање / странично наосување (Roll)	20°	MG 996R
8	Лево Рамо свртување лево- десно (Yaw)	90°	SG 90
9	Лев Лакт	60°	SG 90
10	Лев Зглоб	50°	SG 90
11	Лева Дланка	170°	SG 90
12	Глава Панорама (Pan)	90°	SG 90
13	Глава Навалување (Tilt)	120°	SG 90



Слика 2. Шема на поврзување: Raspberry Pi – PCA9685 (I2C) – Серво мотори

3.4 PWM конверзија на агол

Секој серво мотор се управува со PWM сигнал на 50 Hz. Ширината на пулсот е линеарно поврзана со аголот: $0^\circ = 500 \mu s$, $90^\circ = 1500 \mu s$ и $180^\circ = 2500 \mu s$. PCA9685 прима 12-битни вредности, но неговата библиотека за Python работи на 16-битни работни циклуси (0-65535). [4] Формулата за конверзија е:

```

19     servo_min = 500
20     servo_max = 2500
21
22     def angle_to_pwm(angle):
23         pulse = servo_min + (servo_max - servo_min) * (angle / 180.0)
24         duty_cycle = int(pulse * 65536 / 20000)
25         return duty_cycle

```

Слика 3. Код со формулата за конверзија на PWM сигнал

3.5 3Д печатење на делови

Целокупната физичка структура на роботот е произведена преку 3Д печатење за функционалниот дел од телото и 2Д ласерски сечени дрва за основата. Сите структурните делови се комбинација од готови дизајни и целосно дизајнирани / променети дизајни во Tinkercad и се отпечатени со FDM (Fused Deposition Modeling) технологија.

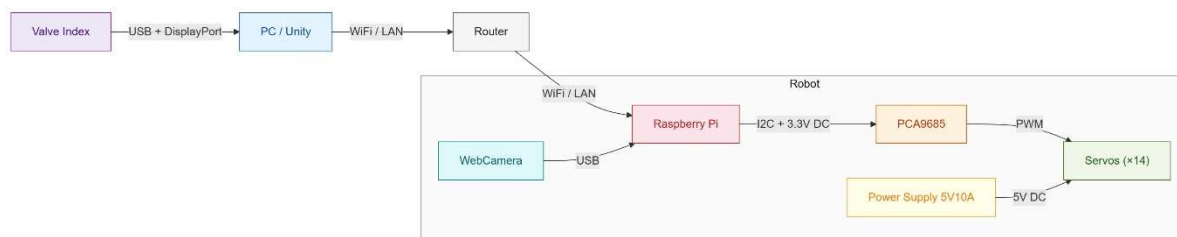
Дизајнот на деловите беше итеративен процес: моделирани и преземени дизајни на држачи за серво моторите, променети и комбинирани по потреба во Tinkercad, потоа испечатен, тестиран за склопување и цврстина, некои делови беа редизајнирани по потреба. Посебно внимание беше посветено на носачите за серво моторите, каде што отворите и монтажните точки мора да бидат на точна позиција за склопување и правилна ориентација на аглиите.

STL датотеките на сите отпечатени делови, заедно со изворните проекти, се достапни за преземање во придружниот репозиториум на проектот. Тоа овозможува целосна репликација на хардверот на роботот. Дел од компонентите беа преземени од јавни репозиториуми како носачите за MG996R [D2], спојките за SG90 [D3] и држачот за камерата [D4].

4. Системска архитектура

4.1 Преглед на системот

Системот е составен од два главни дела кои комуницираат преку локална мрежа: ВР клиент (Unity апликација на компјутер) и хардверски сервер (Python скрипта на Raspberry Pi). Покрај контролниот канал Unity до Raspberry Pi, постои и повратен видео канал Raspberry Pi до Unity.

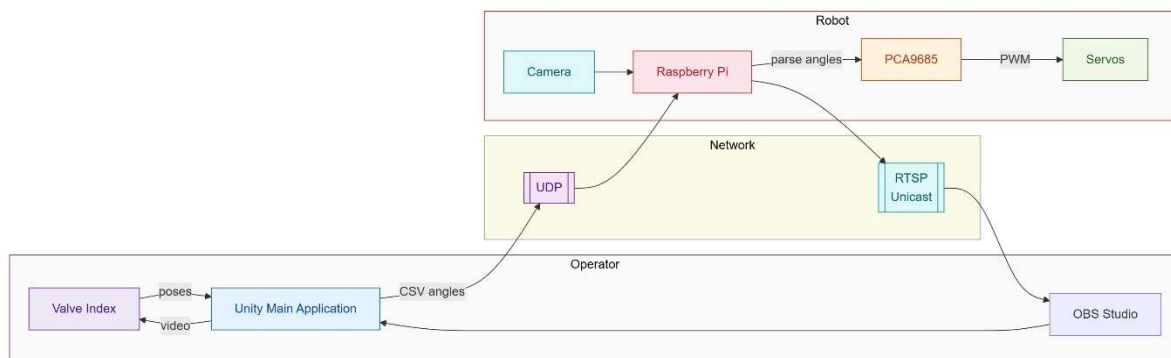


Слика 4. Дијаграм за поврзување на компонентите

4.2 Тек на податоци

Главниот тек на контролни сигнали е следен:

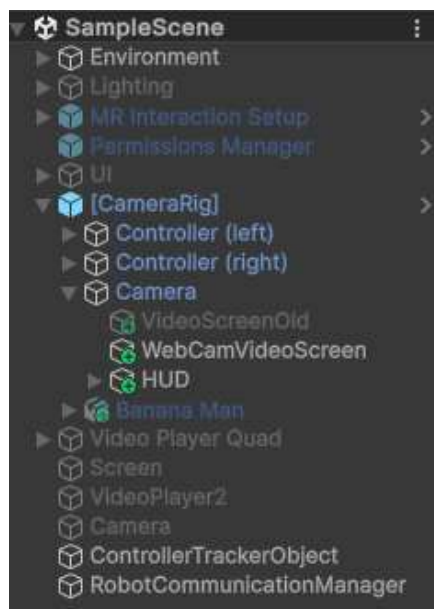
1. Корисникот носи ВР слушалки и држи два ВР контролери.
2. SteamVR ги следи позициите и ориентациите на контролерите и слушалките во реален простор.
3. ControllerTracker функцијата пресметува 12 серво вредности (по 6 за секоја рака) преку инверзна кинематика.
4. HeadTracker пресметува 2 серво вредности (панорама и навалување) од ротацијата на слушалките.
5. RobotCommunicationManager прави агрегација на 14-те вредности и ги испраќа преку UDP socket секои 50 ms.
6. Raspberry Pi ги прима UDP пакетите, ги парсира аглите и ги испраќа преку I2C канал на PCA9685 контролерот.
7. PCA9685 генерира 14 независни PWM сигнали кои ги управуваат серво моторите.
8. Raspberry Pi го испраќа видео сигналот од камерата преку RTSP.
9. На компјутерската страна преку OBS Studio го преземаме видеото и како виртуелна камера го испраќаме на Unity.
10. Unity го прикажува видеото на виртуелната реалност.



Слика 5. Дијаграм за проток на податоци

4.3 Unity сцена и компоненти

Unity сцената содржи неколку клучни GameObject компоненти: CameraRig е SteamVR објект кој содржи објекти за контролерите Controller (left) и Controller (right) и објектот Camera, на кој е прикачена скриптата HeadTracker за следење на движењата на главата. Под Camera се наоѓа WebCamVideoStream, кој го прикажува видео протокот од роботот и HUD кој претставува 3Д UI во VR просторот за приказ на статусот на роботот. Надвор од CameraRig хиерархијата се наоѓаат два дополнителни објекти: ControllerTrackerObject, на кој е прикачена скриптата ControllerTracker за следење на двата контролери и RobotCommunicationManager објект кој е одговорен за комуникацијата со роботот.



Слика 6. Хиерархија на објектите во Unity сцената

5. ВР клиент, Unity имплементација

5.1 Следење на ВР контролери (ControllerTracker)

ControllerTracker.cs е главната скрипта која управува со следење на двата ВР контролери и пресметување на инверзната кинематика. За секоја рамка податоци што се испраќа, ограничено на 20 Hz, се пресметуваат шест серво вредности по контролер. Скриптата ги поддржува истовремено левата и десната рака, со симетрична ИК логика контролирана преку параметарот isLeft.

5.1.1 Позиционирање на рамото

Позицијата на рамото во координатниот систем се пресметува релативно на позицијата на главата (ВР слушалките) кога го притискаме соодветното копче “В” за рекалибрација на рамото. Ова овозможува поставување на рамото при започнување на апликацијата и доколку операторот има потреба да се премести на друга позиција.

```
206  void RecalibrateShoulderPos(bool isLeft)
207  {
208      Transform head = GetHead();
209      if (head == null)
210      {
211          Debug.LogWarning("RecalibrateBasePosition: no headset transform found.");
212          return;
213      }
214
215      if (isLeft)
216      {
217          leftShoulderPos = head.position + MirroredOffset(shoulderOffsetFromHead);
218          leftWristInit = false;
219      }
220      else
221      {
222          rightShoulderPos = head.position + shoulderOffsetFromHead + new Vector3(rightShoulderXOffset, 0f, 0f);
223          rightWristInit = false;
224      }
```

Слика 7. Функција за калибрација на почетните позиции на рамото

5.2 Инверзна кинематика за раце

Инверзната кинематика (ИК) е постапката за пресметување на аглите на зглобовите потребни за да ја достигне целната точка (зглобот/раката) до саканата позиција. [6][7][12] За разлика од директната кинематика (ДК) каде што позицијата се пресметува од дадени агли, ИК го решава обратниот проблем: од дадената позиција ги добиваме потребните

агли. Функцијата `ComputeAndSend()` ги пресметува сите 6 степени на слобода за дадена рака.

5.2.1 Помошни функции

Функцијата `atan2(y, x)` [7] го враќа аголот во радијани помеѓу позитивната X оска и точката (x, y) во рамнината, во опсег $(-\pi, +\pi]$. За разлика од обичниот `atan(y/x)`, `atan2` правилно го одредува квадрантот бидејќи ги зема во предвид знаците на двата аргументи поединечно и никогаш не дели со нула. Во контекстот на ИК, `atan2(target.y, target.z)` го пресметува аголот на векторот “target” во YZ рамнината, односно колку е завртено нагоре или надолу.

5.2.2 Флексија/екстензија на рамо (Pitch) – Канал 0/6

Аголот за наклон го претставува движењето на рамото напред-назад. Се пресметува преку `atan2` на Y и Z компонентите од рамото кон целниот зглоб, изразен во локалниот координатен систем на рамото.

```
252      Vector3 toTargetFromShoulder = targetWristPos - shoulderPos;  
253      float shoulderPitchDeg = Mathf.Atan2(toTargetFromShoulder.y, toTargetFromShoulder.z) * Mathf.Rad2Deg;
```

Слика 8. Пресметка за навалување на рамо (Pitch)

5.2.3 Абдукција/адукција на рамо (Roll) – Канал 1/7

Аголот за валање го претставува страничното движење на рамото. За негово пресметување се применува преместување од 3 cm за да компензираме за големината на физичките серво мотори. [12]

```
255      Vector3 rollPos = shoulderPos + new Vector3(0.03f * mirrorSign, 0f, 0f);  
256      Vector3 toTargetFromRoll = targetWristPos - rollPos;  
257      float shoulderRollDeg = Mathf.Atan2(toTargetFromRoll.x, toTargetFromRoll.z) * Mathf.Rad2Deg;
```

Слика 9. Пресметка за валање на рамо (Roll)

5.2.4 Медијална/латерална ротација на рамо (Yaw) – Канал 2/8

Аголот за свртувањето претставува движењето на раката околу својата оска. За неговото пресметување се применува дополнително поместување од 4 cm напред (Z оска) од позицијата на валање за да компензираме за големината на физичките серво мотори. [12]


```

259 Vector3 pitchPos = rollPos + new Vector3(0f, 0f, 0.04f);
260 Vector3 toTargetFromPitch = targetWristPos - pitchPos;
261 float shoulderYawDeg = Mathf.Atan2(toTargetFromPitch.x, toTargetFromPitch.z) * Mathf.Rad2Deg;

```

Слика 10. Пресметка за свртување на рамо (Yaw)

5.2.5 Свиткување на лакт – Канал 3/9

Аголот на лактот се пресметува преку законот на косинуси, стандардна геометриска техника за триаголник со познати страни. [6] Во триаголникот рамо-лак-зглоб, страните се должините на сегментите на раката.

```

263 Vector3 elbowDir = toTargetFromPitch.normalized;
264 Vector3 elbowPos = pitchPos + elbowDir * 0.03f;
265 float reach = Mathf.Min((targetWristPos - elbowPos).magnitude, forearmLength - 0.001f);
266 float elbowRad = Mathf.Acos(
267     Mathf.Clamp(
268         (forearmLength * forearmLength + 0.03f * 0.03f - reach * reach) /
269         (2 * forearmLength * 0.03f),
270         -1f, 1f
271     )
272 );
273 float elbowDeg = elbowRad * Mathf.Rad2Deg;

```

Слика 11. Пресметка на агол за лакт

5.2.6 Ротација на зглоб – Канал 4/10

Ротацијата на зглобот е најсложениот аспект на ИК. Овде користиме пристап на пресметување ротација на раката (twist extraction)[8] и потоа пресметуваме агол на зглобот спрема ориентацијата на контролерот.

```

345  ✓ float ComputeSmoothWrist(SteamVR_Behaviour_Pose controller, float mirrorSign, Vector3 shoulderPos,
346      ref float prevAngle, ref float smoothedAngle, ref bool initialized)
347  {
348      Vector3 controllerOffset = controller.transform.rotation * controllerToWristOffset;
349      Vector3 targetWristPos = controller.transform.position + controllerOffset;
350      Vector3 rollPos = shoulderPos + new Vector3(0.03f * mirrorSign, 0f, 0f);
351      Vector3 yawPos = rollPos + new Vector3(0f, 0f, 0.04f);
352      Vector3 toTargetFromYaw = targetWristPos - yawPos;
353      Vector3 elbowPos = yawPos + toTargetFromYaw.normalized * 0.03f;
354
355      Vector3 forearmDir = (targetWristPos - elbowPos).normalized;
356      Vector3 controllerUp = controller.transform.up;
357
358      Vector3 twistUp = controllerUp - Vector3.Project(controllerUp, forearmDir);
359      twistUp.Normalize();
360
361      Vector3 referenceUp = Vector3.up - Vector3.Project(Vector3.up, forearmDir);
362      if (referenceUp.magnitude < 0.1f)
363          referenceUp = Vector3.forward - Vector3.Project(Vector3.forward, forearmDir);
364      referenceUp.Normalize();
365
366      float angle = -Vector3.SignedAngle(referenceUp, twistUp, forearmDir);
367      angle *= 0.75f;
368
369      if (!initialized)
370      {
371          smoothedAngle = angle;
372          prevAngle = angle;
373          initialized = true;
374          return angle;
375      }
376
377      float delta = Mathf.DeltaAngle(prevAngle, angle);
378      float target = prevAngle + delta;
379      smoothedAngle = Mathf.Lerp(smoothedAngle, target, 1f - wristSmoothingFactor);
380      prevAngle = smoothedAngle;
381      return smoothedAngle;
382  }

```

Слика 12. Пресметка за ротација и измазнување на движењето на зглобот

5.2.7 Зафат на дланка – Канал 5/11

Аголот на дланката се добива со директна пресметка на сензорот за стисок на SteamVR контролерите кој враќа аналогна вредност од 0 до 1 и ја пресликуваме во серво агол од 0° до 180° со додадена калибрација.

```

282     SteamVR_Action_Single activeGrip = isLeft ? leftGripAction : gripAction;
283     float gripServo = activeGrip != null ? Map(activeGrip.GetAxis(inputSource), 0, 1, 0, 180) : 0f;

```

Слика 13. Пресметка на аголот на зафатот

5.2.8 Калибрациско поместување

По ИК пресметките, пред праќањето на серво вредностите, на секој агол се применуваат калибрациски поместувања. Овие поместувања ги компензираат физичките несовершености во монтажата на серво моторите и разликите помеѓу идеалниот и реалниот нулти агол:

Параметар	Десна рака	Лева рака	Цел
Калибрација на наклон (Pitch)	+10°	-10°	Природна позиција кога рацете се пред градите
Калибрација на валање (Roll)	+60°	-30°	Природна позиција кога рацете се пред градите
Калибрација на зглоб	+60°	-90°	Ротација на зглобот во природна и интуитивна положба
Калибрација на дланка	+20°	+20°	Целосно затворање на дланката при стисок

5.3 Следење на главата

Скриптата HeadTracker.cs ги конвертира ротациите на ВР слушалките во два серво агли за управување со панорамското движење и навалувањето на главата на роботот. При вклучување на роботот, скриптата ја зима позицијата од калибрацијата кога го местиме SteamVR системот и истата може да ја промениме со притискање на копчето “В” за калибрација на системот.

```

164     Vector3 forward = currentTransform.forward;
165     panAngle = Mathf.Atan2(forward.x, forward.z) * Mathf.Rad2Deg;
166     tiltAngle = Mathf.Asin(forward.y) * Mathf.Rad2Deg;

```

Слика 14. Пресметка на аглите на главата

5.4 Управување со комуникацијата

Скриптата `RobotCommunicationManager.cs` е централна функција која ги агрегира сите серво вредности и ги испраќа преку UDP сокет. Singleton шаблонот осигурува дека постои точно еден менаџер во сцената кој сите скрипти можат да пристапат до него преку статичката променлива `RobotCommunicationManager.Instance`.

Менаџерот одржува две низи за серво вредности: агли кон кои движиме (`targetServoValues[]`) и моментални агли (`currentServoValues[]`). Оваа архитектура ја користиме за алгоритмот за заштита на серво моторите од нагли движења и нагли скокови.

```
117         float maxStep = maxDegreesPerSecond * sendInterval;
118         for (int i = 0; i < ServoCount; i++)
119         {
120             float diff = targetServoValues[i] - currentServoValues[i];
121             currentServoValues[i] += Mathf.Clamp(diff, -maxStep, maxStep);
122         }
```

Слика 15. Функција за ограничување на стапката на промена на агол

Серијализација на податоците и испраќање на UDP порака во CSV формат:

```
191         byte[] data = Encoding.ASCII.GetBytes(message);
192         udpClient.Send(data, data.Length, remoteEndPoint);
```

Слика 16. Серијализација и праќање на податоци

5.4.1 Безбедносен механизам

Системот поддржува две нивоа на безбедносна интервенција достапни преку копчето “А” на ВР контролерот. Кратко притискање ги зачувува моменталните агли и го прекинува пресметувањето на нови агли, а долгиот притисок (подолго од 0,5 секунди) ги поставува сите серво мотори во безбедните позиции. Кратко притискање на истото копче додека е прекинато или во безбедните позиции дозволува да продолжиме со користење на роботот.

5.5 Дисплеј за статус

Скриптата `HUDDisplay.cs` е прикачена на `TextMeshPro` виртуелен панел во ВР просторот. Таа го известува операторот за тековната состојба на системот преку бои и текст: “Start” со зелена боја – работна состојба, “Paused” со жолта боја – моторите се застанати во

последната позиција и “Safe” со портокалова боја – моторите се враќаат на безбедна позиција. Дисплејот се наоѓа на горниот десен агол и секогаш е во видното поле на VR слушалките.

5.6 Видео проток

Видеото го примаме преку OBS Studio на компјутерот и го препраќаме како виртуелна веб камера. Со скриптата WebCamDisplay.cs ја читаме виртуелната веб камера и ја прикажуваме на Quad објект во видикот на VR слушалките.

6. Серверска имплементација (Raspberry Pi)

6.1 UDP сервер

Серверската скрипта `vrobot_run.py` на Raspberry Pi има архитектура ориентирана околу настани, без фиксна фреквенција на работа. Скриптата го обработува секој UDP пакет веднаш штом пристигне. Иницијализацијата на скриптата започнува со воспоставување на UDP сокет и I2C врска со PCA9685 контролерот. [5]

```
9      UDP_IP = '0.0.0.0'
10     UDP_PORT = 6000
11
12     sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
13     sock.bind((UDP_IP, UDP_PORT))
14
15     i2c = busio.I2C(SCL, SDA)
16     pca = PCA9685(i2c)
17     pca.frequency = 50
```

Слика 17. Иницијализација на UDP врска и PCA9685 контролер

6.2 Главна јамка за обработка на податоците

Главната јамка го блокира извршувањето на кодот додека не пристигне нов UDP пакет. По приемот, пораката се декодира, 14-те агли се парсираат и директно се препраќаат на PCA9685 каналите. [5]

```

56 while True:
57     try:
58         data, addr = sock.recvfrom(1024)
59     except socket.timeout:
60         continue
61     angles = data.decode().strip().split(',')
62     rPitch, rRoll, rYaw, rElbow, rWrist, rGrip, lPitch, lRoll, lYaw, lElbow, lWrist, lGrip, headx, heady = map(float, angles)
63
64     pca.channels[0].duty_cycle = angle_to_pwm(rPitch)
65     pca.channels[1].duty_cycle = angle_to_pwm(rRoll)
66     pca.channels[2].duty_cycle = angle_to_pwm(rYaw)
67     pca.channels[3].duty_cycle = angle_to_pwm(rElbow)
68     pca.channels[4].duty_cycle = angle_to_pwm(rWrist)
69     pca.channels[5].duty_cycle = angle_to_pwm(rGrip)
70
71     pca.channels[6].duty_cycle = angle_to_pwm(lPitch)
72     pca.channels[7].duty_cycle = angle_to_pwm(lRoll)
73     pca.channels[8].duty_cycle = angle_to_pwm(lYaw)
74     pca.channels[9].duty_cycle = angle_to_pwm(lElbow)
75     pca.channels[10].duty_cycle = angle_to_pwm(lWrist)
76     pca.channels[11].duty_cycle = angle_to_pwm(lGrip)
77
78     pca.channels[12].duty_cycle = angle_to_pwm(headx)
79     pca.channels[13].duty_cycle = angle_to_pwm(heady)

```

Слика 18. Парсирање и испраќање на аглиите на серво моторите

7. Мрежна комуникација

7.1 UDP протокол и формат на пораки

UDP е избран за контролниот канал поради ниското задоцнување и едноставност. [11] Бидејќи системот оперира во реално време, загарантираната испорака на пакети не е приоритет затоа што во случај на загуба, наредниот пакет ги содржи сите потребни информации за актуелната состојба на контролите. Форматот на пораката е низа од 14 вредности во CSV (Comma-separated values) формат, кодирани во ASCII.

Пример порака во безбедна позиција:

“90.0, 160.0, 90.0, 60.0, 130.0, 170.0, 90.0, 20.0, 90.0, 60.0, 50.0, 170.0, 90.0, 120.0”

Типичен пакет зафаќа околу 96 ASCII бајти. При 20 Hz фреквенција, вкупниот мрежен пропусен опсег е $96 \times 20 = 1920$ bytes/s (бајти/секунда) што е од прилика 15 kbit/s (килобити/секунда) е занемарливо мало за секоја WiFi или Ethernet врска.

7.2 Временска анализа и задоцнување

Вкупното задоцнување на системот е сума од неколку компоненти по патеката од ВР контролерот до серво моторот:

Компонента	Типично задоцнување
ВР контролер	13 ms
ИК пресметување	< 1 ms
Бафер во 20 Hz испраќач	до 50 ms
UDP мрежен пренос WiFi	< 1 ms
Raspberry Pi обработка	< 1 ms
PCA9685 I2C запис	< 1 ms
Механички одговор на серво мотор	50–200 ms
ВКУПНО (приближно)	117-267 ms

7.3 RTSP видео проток

RTSP видео протокот се одвива со спротивна насока – од Raspberry Pi кон Unity клиентот. На страната на Raspberry Pi, v4l2rtspserver ја чита видео камерата (/dev/video0) [10] и го емитува на порта 8554. OBS Studio го декодира и го прикажува како виртуелна камера, Unity ја презема виртуелната камера и ја прикажува на екранот на ВР слушалката. Видео

протоколот е ограничен на 1 Mbps (мегабити во секунда) што нуди доволно добар квалитет и не го зафаќа целиот WiFi опсег. [9]

Параметар	Вредност
Протокол	RTSP преку UDP
URL	rtsp://192.168.0.203:8554/unicast
Сервер	v4l2rtspserver на Raspberry Pi
Задоцнување	50-100 ms во нормални услови
Резолуција	640 × 480 пиксели
Слики во секунда	30
Формат	H.264

```
## Start video stream:  
cd v4l2rtspserver  
./v4l2rtspserver -W 640 -H 480 -F 30 -P 8554 -v -b 1000000 /dev/video0
```

Слика 19. Скрипта за започнување на видео протоколот

8. Тестирање и резултати

8.1 Методологија на тестирање

Системот е тестиран во неколку фази.

Функционално тестирање – верификација дека секој серво одговара на ИК пресметките. Секоја оска на движење е тестирана изолирано со поставување на ВР контролерот во граничните позиции. Резултатите од конзолата за дебагирање во Unity беа споредувани со визуелното движење на роботот.

Тестирање на мрежна стабилност – во испраќањето на UDP пораките немаше значајни проблеми со контрола на роботот дури и во ситуации каде што WiFi мрежата имаше 40% доверливост (вредности добиени од мрежниот интерфејс на Raspberry Pi). Во видео протокот со користење на LAN мрежа со кабел немаше загуби и задоцнувањето беше стабилно, додека со користење на WiFi дисторзијата на видеото беше стабилна во совршени услови, но значително се зголемуваше при тестирање со помала доверливост.

Тестирање на безбедносниот механизам – потврдено дека краткото притискање на копчето го пазира системот, долгиот ги враќа серво моторите во безбедни позиции и повторното притискање дозволува да продолжиме со користење на системот.

Субјективна евалуација на задоцнување – земајќи ги во предвид инерцијата на серво моторите и задоцнувањето од видео протокот, визуелниот одговор е приближно 160-280 ms по движењето на корисникот, што е прифатливо за телеоперација.



Слика 20. Демонстрација на управување со роботот - корисникот го контролира системот преку ВР слушалки и контролери

8.2 Резултати

Системот успешно ги исполнува следните функционални барања:

- Следење на двата ВР контролери со 5 степени на слобода + дланка за секоја рака
- ИК пресметување за сите 12 серво канали на рацете во реално време
- Следење на главата со 2 степени на слобода (панорама и навалување)
- UDP пренос на сите 14 вредности на 20 Hz со минимална загуба на пакети
- Директно контролирање на серво моторите преку PCA9685 на Raspberry Pi
- Повратна видео информација преку RTSP проток
- Измазнување на брзината за заштита на физичките делови на роботот
- Безбедносен механизам со две нивоа достапни преку копче на ВР контролерите
- Рекалибрација на почетната позиција на рамото достапно преку копче на ВР контролерите
- Дисплеј за статус на роботот во ВР околината

8.3 Предизвици и ограничувања

Гимбал заклучување на зглобот: Класичниот пристап со Ојлерови агли беше напуштен по откривање на гимбал заклучувањето при специфични позиции на подлактицата. Решен е со пристап за пресметување ротација на раката (twist extraction). Оваа техника е робустна за сите ориентации.

Механичка точност: Серво моторите имаат механичка непрецизност. Ова е доволно за задачи на манипулација но не за прецизни задачи. Калибрациските поместувања делумно ја компензираат оваа непрецизност.

Видео задоцнување: RTSP декодирањето додава дополнителни 50-100 ms задоцнување на видео протокот и имаме дополнителни комплексности со користење на OBS Studio за конвертирање во виртуелна веб камера, а потоа Unity го прикажува на ВР слушалката. Решение за во иднина: Користење WebRTC протокол и примање на видео протокот во Unity користејќи библиотека за да ја намалиме комплексноста на системот и задоцнувањето на видеото.

Мрежна зависност: Системот бара стабилна LAN врска. При загуба на мрежата, Raspberry Pi ги задржува последните серво позиции. Решение за во иднина: Алгоритам кој го враќа роботот во безбедната позиција доколку не пристигне UDP пакет подолго од 1000 ms.

9. Заклучок

9.1 Постигнати резултати

Во рамките на оваа дипломска работа е проектиран, имплементиран и верифициран функционален систем за управување со човеколик робот преку виртуелна реалност. Системот успешно ги поврзува ВР контролерите и слушалката со физичките серво мотори на роботот преку алгоритми за инверзна кинематика и UDP мрежна комуникација.

Клучните придонеси на трудот се: целосна имплементација на инверзна кинематика за две раце и глава за човеколик робот со 14 серво канали, техника за решавање на гимбал заклучувањето при ротација на зглобот, алгоритам за заштита на хардверот со измазнување на движењата, интегриран видео проток со ниско задоцнување за телепресентно искуство и безбедносен механизам со две нивоа управлив директно преку ВР контролерите.

Системот претставува целосна демонстрација на дизајнирање и интеграција помеѓу ВР средината и физичката роботска контрола, со акцент на интуитивноста на управувањето, заштитата на хардверот и ниско задоцнување на контролниот канал.

9.2 Насоки за понатамошна работа

Бинарен протокол: Наместо CSV текст, употреба на бинарен протокол (8 бита по серво $\times$ 14 = 112 бјти, наспроти моменталните 96 бјти) за намалување на мрежните трошоци и зголемување на фреквенцијата на испраќање.

Промена на видео проток: Замена на RTSP со WebRTC протокол за значително намалено задоцнување и користење на библиотека во Unity за директно читање и декодирање на видео протокот наместо користење на OBS Studio.

Редизајн на 3Д принтани делови: Спојување на делови и додавање потребни исечоци во 3Д дизајните наместо користење на лепак и рачно сечење/глодање на деловите. Со ова ќе добиеме зголемена цврстина на целокупниот робот, намалена тежина и полесен пристап за поправка на деловите.

Поддршка на OpenXR и Meta Quest: Миграција на Unity проектот од SteamVR plugin кон OpenXR стандард за постигнување на хардверска независност. Со ова ќе се овозможи употреба на други комплети за ВР паралелно со постојната поддршка на Valve Index, без потреба од одржување на посебни верзии од апликацијата. OpenXR прави апстракција на разликите помеѓу комплетите за ВР преку единствено API, а постојните SteamVR контроли се заменуваат со OpenXR Action System за унифицирано мапирање на влезовите.

Подобрување на алгоритмот за инверзна кинематика: Замена или надградба на тековниот алгоритам за подобрување на природноста и агилноста во движењата на роботот. Со ова

би се овозможиле попрецизни движења на екстремитетите, подобра адаптација на различни пози и поприродно следење на движењата од ВР кон роботот.

Додавање на стерео визија: Додавање на дополнителна камера со цел формирање на стерео пар и постигнување на значително подобрена перцепција на длабочина. Со поставување на две камери на фиксно растојание се овозможува проценка на оддалеченост на објекти, аналогно на принципот на човечкиот бинокуларен вид. Резултатот е подобрена просторна ориентација при управување преку ВР интерфејсот.

Литература

- [1] Valve Corporation, “SteamVR Unity Plugin – Documentation and API Reference,” GitHub Repository, 2023. [Online]. Достапно на: https://github.com/ValveSoftware/steamvr_unity_plugin
- [2] Unity Technologies, “Unity 6 Manual – Universal Render Pipeline,” Unity Documentation, 2024. [Online]. Достапно на: <https://docs.unity3d.com/6000.4/Documentation/Manual/UnityManual.html>
- [3] Raspberry Pi Foundation, “Raspberry Pi 4 Model B – Technical Specifications,” Raspberry Pi Documentation, 2024. [Online]. Достапно на: <https://www.raspberrypi.com/documentation/>
- [4] NXP Semiconductors, “PCA9685 – 16-channel, 12-bit PWM Fm+ I2C-bus LED Controller,” Product Datasheet, Rev.4, 2015.
- [5] Adafruit Industries, “Adafruit CircuitPython PCA9685 Library,” CircuitPython Library Documentation, 2023. [Online]. Достапно на: <https://docs.circuitpython.org/en/latest/README.html>
- [6] J. J. Craig, “Introduction to Robotics: Mechanics and Control,” 3rd ed. Upper Saddle River, NJ, USA: Pearson Prentice Hall, 2005.
- [7] B. Siciliano, L. Sciavicco, L. Villani, & G. Oriolo, Robotics: Modelling, Planning and Control. London, UK: Springer, 2009.
- [8] J. B. Kuipers, Quaternions and Rotation Sequences. Princeton, NJ, USA: Princeton University Press, 1999.
- [9] H. Schulzrinne, A. Rao, R. Lanphier, M. Westurlund, M. Stiemerling, “Real Time Streaming Protocol Version 2.0 (RTSP/2.0),” IETF, RFC 7826, Dec. 2016.
- [10] H. Verkuil, “ELC: An Introduction to the Video4Linux Framework,” Cisco Blogs – Open at Cisco, Mar. 27, 2014. [Online]. Достапно на: <https://blogs.cisco.com/openatcisco/elc-an-introduction-to-the-video4linux-framework>
- [11] F. T. Al-Dhief, N. Sabri, M. Albadr, “Performance Comparison between TCP and UDP Protocols in Different Simulation Scenarios,” International Journal of Engineering and Technology, 2018.
- [12] R.M. Murray, Z. Li, and S.S. Sastry, A Mathematical Introduction to Robotic Manipulation. Boca Raton, FL, USA: CRC Press, 1994.

Додаток А: Преземени 3Д модели

Следниве 3Д модели беа преземени од јавни репозиториуми и употребени (целосно или делумно) при изградбата на физичкиот робот. Секој модел е под Creative Commons Attribution лиценца.

- [D1] yisparyan. Robot Gripper 9g Micro Servo. Thingiverse. Достапно на: <https://www.thingiverse.com/thing:715525/comments>
- [D2] Dingo_aus. Hobby Servo Holder for MG996R. Printables. Достапно на: <https://www.printables.com/model/11782-hobby-servo-holder-for-mg996r>

[D3] Bernardus Rendy. SG90 Servo Bracket. Grabcad. Достапно на: <https://grabcad.com/library/sg90-servo-bracket-1>

[D4] tekyinblack. SG90 MG90 Servo Pan and Tilt Camera platform. Thingiverse. Достапно на: <https://www.thingiverse.com/thing:6008663>

Додаток Б: Хардверски компоненти

Следниве хардверски компоненти беа употребени при изградбата и работата на системот. Секоја компонента е наведена со официјалната страница за продуктот за техничка референца и верификација.

[H1] Raspberry Pi Ltd. (2019). Raspberry Pi 4 Model B. Микро компјутер употребен како серверски дел на системот за прием на UDP пакети и управување со PCA9685 контролерот. Достапно на: <https://www.raspberrypi.com/products/raspberry-pi-4-model-b/>

[H2] NXP Semiconductors. (2015). PCA9685 – 16-Channel, 12-Bit PWM Fm+ I2C-Bus LED Controller. PWM контролер поврзан со Raspberry Pi преку I2C за управување до 16 серво мотори паралелно. Достапно на: <https://www.nxp.com/products/power-drivers/lighting-driver-and-controller-ics/led-drivers/16-channel-12-bit-pwm-fm-plus-ic-bus-led-driver:PCA9685>

[H3] Valve Corporation. (2019). Valve Index VR Kit. Steam Store. ВР слушалки и контролери употребени за следење на движењата на корисникот и пренесување на ИК вредностите до роботот. Достапно на: <https://store.steampowered.com/valveindex>

Додаток В: Изворен код и демонстрација

GitHub: <https://github.com/JovanNikolov/VRobotThesis>

Демонстрациско видео: <https://youtu.be/ArwLvA5CuVU>